IVDC Autonomy Induction 2026

Part A: Research / Learning

PS 1 Computer Vision | PS 2 Motion Planning | PS 3 Controllers / RL

Name: _____ GitHub repo: _____

Answer numbering follows the problem statements: **PS.Gate.Question** for PS 1 and PS 3 (e.g. 1.2.3), and **PS 2.Topic.Bullet** for PS 2. References are listed at the end of each PS.

PS 1: Computer Vision

Gate 1: How does a computer see?

1.1.1 What is a digital image? Pixel, resolution, matrix representation

A digital image is a grid of tiny samples of light. Each sample is a **pixel** (picture element) holding a number (or a few numbers) for brightness/colour at that location. **Resolution** is the number of pixels, written width x height (e.g. 1920 x 1080); more pixels capture more detail but cost more memory and compute. Because the pixels sit on a regular grid, an image is just a **matrix**: a grayscale image is an H x W array of values (0 = black, 255 = white for 8-bit), and a colour image is an H x W x 3 array (one matrix per channel). In OpenCV/NumPy this means `img[row, col]` reads a pixel and image processing is arithmetic on arrays. Note OpenCV loads colour in BGR order, not RGB.

1.1.2 RGB vs grayscale; why process them differently?

RGB stores three channels (red, green, blue) per pixel, so colour is available. **Grayscale** stores one intensity channel (a weighted sum of R, G, B). Grayscale is 3x smaller and faster to process, and is enough for tasks that depend on brightness structure such as edges. RGB (or HSV) is needed when colour is the cue, such as yellow vs white lane lines, traffic lights, or green grass vs grey road. A vehicle has a limited real-time compute budget, so it uses grayscale for fast geometry steps and colour only where it adds information.

Gate 2: Basic image processing

1.2.1 Gaussian blur and why use it before feature detection

Gaussian blur replaces each pixel with a weighted average of its neighbours, where weights follow a Gaussian (bell) curve, so closer pixels count more. It suppresses high-frequency noise and tiny texture (e.g. gravel on asphalt). Edge detectors respond to sudden intensity changes, and noise creates many small false ones, so blurring first gives cleaner, more meaningful edges. Too large a kernel/sigma also blurs real edges, so it is a trade-off.

1.2.2 HSV colour space and why it can beat RGB for lane detection

HSV describes colour as **Hue** (the colour type), **Saturation** (how pure vs washed out) and **Value** (brightness). In RGB, brightness and colour are mixed across all three channels, so a shadow changes R, G and B together. In HSV, brightness is separated into V, so thresholds on H and S stay fairly stable under lighting changes. White lines = low S and high V; yellow lines = hue near yellow with high S. This makes colour thresholding more robust and easier to tune. (In OpenCV, H runs 0-179, S and V 0-255.)

1.2.3 What does Canny edge detection do?

Canny finds thin, well-localised edges in four steps: (1) smooth with Gaussian blur; (2) compute intensity gradients (Sobel) giving magnitude and direction; (3) **non-maximum suppression**, keeping only local maxima along the gradient direction so edges are 1 pixel thick; (4) **hysteresis thresholding** with two thresholds: pixels above the high threshold are strong edges, those between low and high are kept only if connected to a strong edge, and the rest are discarded. Output is a binary edge map.

1.2.4 Region of Interest (ROI) and why it reduces false detections

An ROI is a sub-area of the image (usually a polygon or trapezoid covering the road ahead) where we look for lanes; everything else is masked to zero. The sky, trees, buildings and far-off clutter produce edges that look like lines but are not lane markings. Masking them out removes those false candidates and also reduces computation. The assumption is that the camera is fixed, so the road always occupies a roughly known region.

Gate 3: Detecting structure

1.3.1 What is a contour and what can it tell us?

A contour is a curve joining continuous boundary points of the same colour/intensity, found on a binary image (`cv2.findContours`). From a contour we can get area, perimeter, bounding box, centroid (via moments), orientation, convex hull, and an approximate polygon shape (`approxPolyDP`). In lane work, contour area and shape can be used to keep long thin lane-like blobs and reject small noise specks.

1.3.2 Hough Transform and how it finds lane lines

The Hough Transform detects lines by **voting**. A line can be written as r = x cos(theta) + y sin(theta). Each edge pixel (x, y) lies on infinitely many lines, which correspond to a sinusoid of (r, theta) values in **parameter space**. All edge pixels on the same straight line produce sinusoids that intersect at one (r, theta) point, so that cell collects many votes. Peaks in the accumulator (above a threshold) are the detected lines. `cv2.HoughLinesP` (probabilistic version) returns line segments with endpoints. For lanes: run Canny, apply ROI, run HoughLinesP, then separate segments into left/right by slope sign, average each group, and extrapolate to get two lane boundary lines.

Gate 4: Lane / path perception

1.4.1 Why is lane/path detection useful for an autonomous ground vehicle?

It tells the vehicle where the drivable corridor is and where it sits within it. This gives the lateral offset and heading relative to the lane, which feed the controller for lane keeping, supports path planning and safe-area estimation, and works where there are no maps or GPS is poor.

1.4.2 Why do shadows, texture, lighting, missing markings or clutter cause false detections?

- **Shadows** create strong, straight, high-contrast edges (tree or pole shadows) that look like lane lines.
- **Road texture/cracks/tar patches** and kerbs produce extra edges that Hough can group into false lines.
- **Poor/changing lighting** (glare, night, overexposure) shifts brightness so fixed thresholds miss real markings or accept the wrong things.
- **Missing or worn markings** give no edges for the detector to vote on.
- **Clutter** (vehicles, fences, grass boundaries, signs) adds structure that is not a lane.

1.4.3 Why is lane detection alone not sufficient for full autonomy?

Lanes are only one part of perception. A vehicle must also detect and track obstacles, pedestrians and other vehicles, read signs and signals, handle roads without markings (off-road, junctions, construction), localise itself, and then plan and control its motion. Lane detection is also fragile in the situations listed above, so it should be combined with other sensors (LiDAR, radar, depth, GPS/IMU) and with planning and safety logic.

Gate 5: Perspective and BEV

1.5.1 What is perspective distortion?

A camera projects the 3D world onto a 2D image, so objects farther away appear smaller and parallel lines in the world (like lane edges) appear to converge toward a vanishing point. Real distances and angles are therefore not preserved in the image: the same 3.5 m lane width covers many pixels near the camera and few far away.

1.5.2 What is a Bird's-Eye View (BEV) and why does it help?

BEV is a top-down view of the ground plane, obtained by a perspective (homography) transform (`cv2.getPerspectiveTransform` + `warpPerspective`) from four points on the road to a rectangle. In BEV, parallel lane lines look parallel, lane width is roughly constant, and pixel distances map linearly to metres. This makes it easy to fit curves, measure lateral offset and curvature, build occupancy/cost maps, and feed planners and controllers, which also work in top-down coordinates.

PS 1 References

- OpenCV-Python tutorials: Getting Started with Images; Basic Operations on Images; Smoothing Images; Changing Colorspaces; Canny Edge Detection; Contours; Hough Line Transform; Geometric Transformations. docs.opencv.org
- PyImageSearch: OpenCV Smoothing and Blurring; OpenCV Edge Detection (cv2.Canny). pyimagesearch.com

PS 2: Motion Planning

Topic 1: Path planning fundamentals

2.1.1 Path vs trajectory

A **path** is a purely geometric route, an ordered set of positions from start to goal, with no notion of time. A **trajectory** is a path plus timing: it says where the vehicle should be at each time, so it includes velocity, acceleration and heading limits. A planner may output a path; a trajectory generator then makes it drivable by the real vehicle.

2.1.2 Key terms

Term	Meaning
Waypoint	An intermediate point on the path that the vehicle should pass through.
Obstacle	Anything the vehicle must not collide with (walls, cars, pedestrians).
Free space	The set of positions/cells where the vehicle can safely be.
Start state	The vehicle's initial position (and possibly heading/velocity).
Goal	The target position/state the planner must reach.

Topic 2: Environment representation

2.2.1 What is a 2D grid map?

The world is divided into equal square cells in a 2D array. Each cell stores a value describing that patch of space, and the planner moves between neighbouring cells (4-connected or 8-connected). It is simple to implement, and fits array tools such as NumPy.

2.2.2 What is an occupancy grid?

A grid map where each cell stores whether it is occupied, often as a probability of occupancy (0 to 1) updated from sensor data. It is the standard way robots represent obstacles around them.

2.2.3 Free, occupied and unknown cells

- **Free**: known to be empty; the planner may move through it.
- **Occupied**: contains an obstacle; the planner must avoid it.
- **Unknown**: not yet observed (occupancy probability near 0.5). Planners usually treat it as blocked (safe) or as high cost, and explore to reveal it.

Topic 3: Search algorithms

2.3.1 BFS (Breadth-First Search)

BFS expands the start cell, then all its neighbours, then their neighbours, and so on, level by level, using a **FIFO queue**. Intuition: a wave spreading outwards equally in all directions. With equal step costs, the first time it reaches the goal is via the fewest steps, so BFS gives the shortest path in number of moves. It explores many cells because it has no sense of direction toward the goal.

2.3.2 Dijkstra's algorithm and how it differs from BFS

Dijkstra uses a **priority queue** keyed by cost-so-far g(n) and always expands the cell with the lowest total cost from the start. When all moves cost the same it behaves like BFS. When costs differ (e.g. mud = 5, road = 1, or diagonal = 1.414), BFS only minimises number of steps and can return an expensive route, whereas Dijkstra minimises the total cost, so it may take a longer but cheaper path. It requires non-negative costs.

2.3.3 A* and the role of the heuristic

A* also uses a priority queue but orders cells by **f(n) = g(n) + h(n)**, where g is the cost from the start and h is a heuristic estimate of the remaining cost to the goal (e.g. Manhattan distance on a 4-connected grid, Euclidean or octile for 8-connected). The heuristic biases the search toward the goal so fewer cells are explored. If h is **admissible** (never overestimates) A* still returns an optimal path. h = 0 reduces A* to Dijkstra; an over-large h makes it greedy and faster but possibly suboptimal.

	BFS	**Dijkstra**	**A***
Data structure	FIFO queue	Priority queue by g	Priority queue by g + h
Edge costs	Equal only	Any non-negative	Any non-negative
Optimal?	Yes (fewest steps)	Yes (least cost)	Yes if h admissible
Exploration	Many cells, all directions	Many cells, cost-ordered	Fewer, goal-directed

Topic 4: Global and local planning

2.4.1 Global path vs local motion decision

A **global planner** uses the whole known map to compute a complete route from start to goal (e.g. A* on a map). A **local planner** works over a short horizon with live sensor data, deciding the immediate steering/speed to follow the global path while avoiding nearby obstacles and respecting vehicle dynamics. Global = where to go overall; local = what to do right now.

2.4.2 Why re-plan when a new obstacle appears?

Maps are never complete or static: a pedestrian, a stopped car or debris can block the old path or make it unsafe. The vehicle must update its map and re-plan (fully, or by locally repairing the route) to find a new collision-free path, otherwise it would follow an outdated plan into an obstacle.

PS 2 Part B: Visualization (to be completed with your own code)

Add here: a brief explanation of how your visualization works (colours used for free, obstacle, start, goal, explored and path), screenshots, and your observations (e.g. BFS vs A explored-cell count and path length).*

PS 2 References

- Red Blob Games: Introduction to the A* Algorithm; Implementation of A*. redblobgames.com
- Gabriel Gambetta: Pathfinding Demystified, Part I. gabrielgambetta.com

PS 3: Controllers / Reinforcement Learning

Gate 1: Feedback and error

3.1.1 Reference/setpoint, measured value, feedback, error

- **Reference / setpoint**: the desired value (e.g. stay 0 m from lane centre, hold 10 m/s).
- **Measured value**: the actual value from sensors (e.g. current lateral offset or speed).
- **Feedback**: sending the measured value back to compare against the setpoint.
- **Error**: e = reference minus measured value; the controller acts to drive it to zero.

3.1.2 Why is feedback better than blindly executing a precomputed command?

A precomputed (open-loop) command assumes a perfect model and a perfect world. Real vehicles face wheel slip, slopes, wind, battery/motor changes and sensor-free disturbances, so small errors accumulate unnoticed. Feedback (closed-loop) measures the actual result and corrects continuously, so it rejects disturbances and tolerates model errors. Like steering a bicycle, you constantly correct what you observe.

Gate 2: P controller

3.2.1 Proportional control u = Kp * e

The control output u is proportional to the current error: large error gives a large correction, small error a small one. Kp is the gain. Example: steering angle = Kp x lateral error.

3.2.2 Effect of increasing / decreasing Kp

- **Higher Kp**: faster response and smaller error, but more overshoot and oscillation; too high becomes unstable.
- **Lower Kp**: smoother and safer, but slow and sluggish, and leaves a larger remaining error.

3.2.3 Why can a P controller have steady-state error?

A P controller only produces output when error is non-zero. If the system needs a constant non-zero output to hold the setpoint (e.g. throttle to maintain speed against drag or a hill), then u = Kp*e requires e to stay non-zero. The system settles where Kp*e exactly balances the disturbance, leaving a leftover offset. Raising Kp reduces but cannot remove it, and adding integral action fixes it.

Gate 3: PID controller

3.3.1 PID: u = Kp*e + Ki*(integral of e dt) + Kd*(de/dt)

The output is the sum of three terms acting on the error: proportional (present), integral (accumulated past) and derivative (rate of change, predicting the future). Kp, Ki and Kd are tuned gains.

3.3.2 Intuitive role of P, I and D

Term	Intuition	Effect	Too much
P	How big is the error now?	Main driving force; speeds up response	Overshoot, oscillation
I	How long has the error persisted?	Removes steady-state error	Windup, slow oscillation, overshoot
D	How fast is error changing?	Damps motion, reduces overshoot	Amplifies sensor noise, jitter

Gate 4: Path tracking

3.4.1 Cross-track error

The perpendicular (lateral) distance from the vehicle (usually its reference point, e.g. front axle) to the closest point on the desired path. It measures how far to the side of the path the vehicle is.

3.4.2 Heading error

The difference between the vehicle's heading angle and the path's tangent direction at the nearest point. It tells how misaligned the vehicle is with the path direction.

3.4.3 Why do both matter for smooth tracking?

Cross-track error alone ignores direction: a vehicle can be on the path but pointing away, and will soon leave it; or be off the path but already pointing back toward it. Heading error alone ignores position, so the vehicle could run parallel to the path but offset. Using both (as in the Stanley controller, steering = heading error + arctan(k * cross-track error / speed)) lets the vehicle correct position and orientation together, avoiding zig-zagging.

Gate 5: Introduction to RL

3.5.1 Agent, environment, state, action, reward, policy

- **Agent**: the learner/decision-maker (the vehicle's controller).
- **Environment**: everything the agent interacts with (road, map, physics).
- **State (s)**: description of the current situation (e.g. position, speed, lateral error).
- **Action (a)**: a choice the agent can make (steer left/right, accelerate, brake).
- **Reward (r)**: a scalar feedback signal saying how good the last action was (+ for progress, - for collision).
- **Policy (pi)**: the agent's strategy, a mapping from state to action; the goal of RL is to learn a policy that maximises long-term cumulative reward.

3.5.2 What does Q(s, a) represent?

The expected total (discounted) future reward if the agent takes action a in state s and then follows its policy afterwards. A high Q means that action is good from that state. In Q-learning the agent keeps a table of Q-values and updates it with Q(s,a) <- Q(s,a) + alpha*[r + gamma*max over a' of Q(s',a') - Q(s,a)], where alpha is the learning rate and gamma the discount factor. The policy is then to pick the action with the highest Q (with some random exploration during training).

PS 3 References

- MathWorks: What Is PID Control? mathworks.com
- Control Strategies for Autonomous Vehicles (Stanley controller section). arxiv.org
- Hugging Face Deep RL Course, Unit 2: Introduction to Q-Learning. huggingface.co
- Sutton and Barto, Reinforcement Learning: An Introduction (2nd ed.)

Acknowledgment

I used Claude (an AI assistant made by Anthropic) to help draft the Part A research answers in this document. I have reviewed all of the answers, and I understand them and can explain them myself. Other sources I used for learning are listed in the references at the end of each problem statement.